অধ্যায় ১০

Asynchronous JavaScript: Callbacks, Promises, async/await এবং Event Loop

শেখার লক্ষ্য

এই অধ্যায় শেষে তুমি Synchronous ও Asynchronous Execution-এর পার্থক্য, Callbacks, Promise State, `async/await`, Event Loop, Microtask ও Task Queue, `fetch()`, HTTP Error, `AbortController`, Concurrent Requests এবং User-Friendly Loading/Error States ব্যাখ্যা ও প্রয়োগ করতে পারবে। Practical Project: **User Directory API Explorer**।

১০.১ JavaScript কি Single-Threaded?

সাধারণ Browser JavaScript Execution Context-এ একটি Main Thread এক সময়ে একটি JavaScript Call Stack চালায়। কিন্তু Browser-এর Network, Timer, DOM Event-এর মতো সুবিধা Asynchronous কাজ পরিচালনা করতে পারে। Node.js-এও Event Loop এবং Host APIs আছে; CPU-intensive JavaScript Main Thread আটকে দিতে পারে। **Single-threaded JavaScript মানে সব I/O কাজ Blocking নয়।**

```javascript
console.log('A');
setTimeout(() => console.log('C'), 0);
console.log('B');
// A, B, C
```

`setTimeout(..., 0)` সঙ্গে সঙ্গে Callback চালায় না; নির্ধারিত সময়ের আগে নয়, Event Loop সুযোগ পেলে Timer Task চলবে। Real scheduling Browser/Node Host-এর ওপর নির্ভর করে।

১০.২ Call Stack, Web APIs ও Event Loop

একটি Function Call Stack-এ Push হয়; Return করলে Pop হয়। `fetch()` Network Request শুরু করে এবং সঙ্গে সঙ্গে Promise দেয়। Response Ready হলে সংশ্লিষ্ট Promise Reaction Microtask Queue-তে Schedule হয়। Stack খালি হলে Host Microtask Checkpoint-এ Microtasks চালায়। Browser-এ Rendering-এর সুযোগ সাধারণত Task-এর মাঝামাঝি আসে; দীর্ঘ JavaScript Loop UI Freeze করতে পারে।

```
JavaScript Call Stack
        |
        v
  Host APIs (timer / fetch / events)
        |
        +--> Task queue (timers, events)
        +--> Microtask queue (Promise reactions)
        |
        v
 Event Loop schedules execution
```

এই Diagram একটি Conceptual Model, Browser Specification-এর পূর্ণ Scheduler নয়।

১০.৩ Callbacks: প্রথম Asynchronous Pattern

JAVASCRIPT

```javascript
function afterDelay(message, callback) {
  setTimeout(() => callback(null, message), 300);
}
afterDelay('Hello', (error, value) => {
  if (error) return console.error(error);
  console.log(value);
});
```

Callback একটি Function যা অন্য Function-কে দেওয়া হয়। সব Callback Asynchronous নয়: `array.map(callback)` Synchronous। Nested Callbacks বেশি হলে Control Flow ও Error Handling জটিল হতে পারে।

১০.৪ Promise: Future Result-এর Representation

Promise-এর তিনটি State: **pending**, **fulfilled**, **rejected**। Fulfilled বা Rejected হলে Promise Settled। `then()` Success, `catch()` Rejection এবং `finally()` Cleanup Handle করে। `then()` সবসময় নতুন Promise দেয়, তাই Chaining সম্ভব।

JAVASCRIPT

```javascript
const task = new Promise((resolve) => {
  setTimeout(() => resolve(42), 200);
});
task.then(value => value * 2)
    .then(value => console.log(value)) // 84
    .catch(error => console.error(error))
    .finally(() => console.log('finished'));
```

`throw` করলে Promise Chain Rejected হয়। `catch()`-এর ভেতরে Error Return করলে Chain Recover করে; পুনরায় Throw করলে Rejection বজায় থাকে। Unhandled Rejection এড়াতে যথাযথ Error Handling করো।

১০.৫ `async` এবং `await`

`async` Function সবসময় Promise Return করে। `await` ওই Async Function-এর Execution Pause করে, পুরো JavaScript Thread নয়। Rejected Promise `await` করলে Exception-এর মতো আচরণ করে, তাই `try/catch` ব্যবহার করা যায়।

```javascript
async function getGreeting() {
  const value = await Promise.resolve('Welcome');
  return `${value} to JavaScript Mastery`;
}
getGreeting().then(console.log);
```

`await` শুধু Async Function বা Module-এর Top Level-এ বৈধ (সমর্থিত Module Environment-এ)। `forEach(async () => ...)`-এর Return Promises `forEach` অপেক্ষা করে না। Sequential Loop চাইলে `for...of` + `await`, Concurrent কাজ চাইলে `Promise.all(items.map(async ...))`।

১০.৬ Microtask বনাম Task: Output Prediction

```javascript
console.log('1');
setTimeout(() => console.log('4'), 0);
Promise.resolve().then(() => console.log('3'));
console.log('2');
// 1, 2, 3, 4
```

কারণ Synchronous Statements আগে চলে, তারপর Promise Reaction Microtask, তারপর Timer Task। `queueMicrotask()` দিয়েও Microtask Schedule করা যায়। অতিরিক্ত Microtasks Schedule করতে থাকলে Rendering ও অন্য Tasks Starve হতে পারে।

Interactive Thought Experiment

```javascript
async function demo() {
  console.log('B');
  await Promise.resolve();
  console.log('D');
}
console.log('A');
demo();
console.log('C');
// A, B, C, D
```

কেন? `await`-এর পরের অংশ Promise Reaction হিসেবে পরে চলে। এটি বোঝা Debugging-এর জন্য গুরুত্বপূর্ণ।

১০.৭ `fetch()` দিয়ে API Call

`fetch()` Network Failure বা Abort-এর জন্য Reject করতে পারে; কিন্তু HTTP `404`/`500` Response এলে সাধারণত Promise Fulfill হয়। তাই `response.ok` Check করতে হবে। JSON Parsing-ও Fail করতে পারে।

```javascript
async function loadUsers() {
  const response = await fetch('https://jsonplaceholder.typicode.com/users');
  if (!response.ok) throw new Error(`HTTP ${response.status}`);
  const data = await response.json();
  if (!Array.isArray(data)) throw new TypeError('Unexpected API response');
  return data;
}
```

Production Note: Public Demo API Availability, Rate Limits, CORS, Privacy ও Network Reliability পরিবর্তিত হতে পারে। Secrets Frontend JavaScript-এ Embed করা যাবে না।

১০.৮ Request Cancellation ও Race Condition

User Search বা Reload দ্রুত বদলালে পুরোনো Request-এর Response পরে এসে নতুন Data Overwrite করতে পারে। `AbortController` আগের Request Cancel করতে সাহায্য করে। তবে Abort হওয়ার আগেই Response Settle হলে Request-ID Check-ও দরকার হতে পারে।

```javascript
let controller;
let latest = 0;
async function searchRemote(query) {
  controller?.abort();
  controller = new AbortController();
  const requestId = ++latest;
  try {
    const url = `/api/search?q=${encodeURIComponent(query)}`;
    const response = await fetch(url, { signal: controller.signal });
    if (!response.ok) throw new Error(`HTTP ${response.status}`);
    const result = await response.json();
    if (requestId !== latest) return;
    console.log(result);
  } catch (error) {
    if (error.name !== 'AbortError') console.error(error);
  }
}
```

এই Snippet-এ `/api/search` একটি উদাহরণ Endpoint, বাস্তব Project-এর Dependency নয়।

১০.৯ Sequential বনাম Concurrent Work

```javascript
// Sequential: একটি শেষ হওয়ার পরে আরেকটি
const first = await fetch('/api/first');
const second = await fetch('/api/second');

// Concurrent: উভয় Request শুরু, তারপর সব শেষ হওয়ার অপেক্ষা
const [a, b] = await Promise.all([
  fetch('/api/first'), fetch('/api/second')
]);
```

দুটি Request Independent হলে Concurrent Pattern Latency কমাতে পারে। `Promise.all()` একটি Promise Reject হলে দ্রুত Reject করে, তবে অন্য Operation স্বয়ংক্রিয়ভাবে Cancel হয় না। `Promise.allSettled()` সব ফলাফল (Fulfilled/Rejected) সংগ্রহ করে। `Promise.race()` প্রথম Settled Result দেয়; `Promise.any()` প্রথম Fulfilled Result দেয়, সব Reject হলে `AggregateError` দেয়।

API	আচরণ	ব্যবহার
`Promise.all`	সব Fulfill হলে Result, প্রথম Rejection-এ Reject	সব ফলাফল আবশ্যক
`Promise.allSettled`	সব Settle হওয়া পর্যন্ত অপেক্ষা	Partial Failure গ্রহণযোগ্য
`Promise.race`	প্রথম Settled Promise-এর ফল	Timeout Pattern-এর অংশ
`Promise.any`	প্রথম Fulfilled Result	বিকল্প Service/Replica

১০.১০ Retry, Timeout ও Backoff

সব Error-এ Retry উপযুক্ত নয়। `GET`-এর মতো Idempotent Request এবং Transient `429/503` Error-এ সীমিত Retry বিবেচ্য। Payment বা Booking Create-এর মতো Non-idempotent Action অন্ধভাবে Retry করলে Duplicate Transaction হতে পারে। `AbortSignal.timeout(ms)` আধুনিক Environment-এ Timeout Signal দিতে পারে; Compatibility যাচাই করতে হবে। `Promise.race()`-এ Timeout দিলেই Underlying Network Request Cancel হয় না।

```javascript
async function fetchWithTimeout(url, ms = 5000) {
  const controller = new AbortController();
  const timer = setTimeout(() => controller.abort(), ms);
  try {
    const response = await fetch(url, { signal: controller.signal });
    if (!response.ok) throw new Error(`HTTP ${response.status}`);
    return response;
  } finally {
    clearTimeout(timer);
  }
}
```

১০.১১ Practical Project — User Directory API Explorer

Project Goal

একটি Responsive Browser Application, যেখানে Public API থেকে Users Load হবে; Loading, Error, Reload, Search, Sorting ও Cancellation Support থাকবে। Source Code বইয়ের Companion Source Code-এর `source-code/chapter-10/` Directory-তে রয়েছে।

Project Structure

```
source-code/chapter-10/
├── index.html
├── style.css
├── app.mjs
├── directory-core.mjs
└── directory-core.test.mjs
```

Running the Project

1. `source-code/chapter-10` Directory-তে Terminal খোলো।
2. `python -m http.server 8000` চালাও (অথবা VS Code Live Server ব্যবহার করো)।
3. Browser-এ `http://localhost:8000` খোলো। ES Modules-এর জন্য Local HTTP Server ব্যবহার করা সুবিধাজনক।
4. Internet Connection থাকলে JSONPlaceholder Demo API থেকে Users আসবে।
5. Search, Sort ও Reload পরীক্ষা করো; DevTools Network Tab-এ Request Observe করো।
6. Tests: `node --test directory-core.test.mjs` (Node.js 18+ Recommended)।

HTML: `index.html`

HTML

```html
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width,initial-scale=1">
  <title>User Directory API Explorer</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <main>
    <header>
      <h1>User Directory API Explorer</h1>
      <p>Async JavaScript • Fetch • AbortController • Search</p>
    </header>
    <form id="controls" onsubmit="return false">
      <label>Search <input id="search" type="search" placeholder="Name, email, city"></label>
      <label>Sort by
        <select id="sort"><option value="name">Name</option><option value="email">Email</option>
<option value="city">City</option></select>
      </label>
      <button id="reload" type="button">Reload users</button>
```

```
    </form>
    <p id="status" role="status" aria-live="polite">Loading…</p>
    <ul id="users" aria-label="Users"></ul>
  </main>
  <script type="module" src="app.mjs"></script>
</body>
</html>
```

CSS ও JavaScript Files

`style.css`, `app.mjs`, `directory-core.mjs`-এর সম্পূর্ণ, Executable Source Code Project Directory-তে দেওয়া আছে। এখানে গুরুত্বপূর্ণ অংশগুলো ব্যাখ্যা করছি।

JAVASCRIPT

```javascript
// directory-core.mjs
export async function fetchUsers({ fetchImpl = fetch, signal,
  url = 'https://jsonplaceholder.typicode.com/users' } = {}) {
  const response = await fetchImpl(url, {
    signal,
    headers: { Accept: 'application/json' }
  });
  if (!response.ok) throw new Error(`HTTP ${response.status}: users unavailable`);
  const data = await response.json();
  if (!Array.isArray(data)) throw new TypeError('Expected an array of users');
  return data.map(normalizeUser);
}
```

`fetchImpl` Inject করার ফলে Real Network ছাড়াই Mock Function দিয়ে Unit Test করা যায়। `normalizeUser` API Response-এর Field Shape Verify করে। `filterUsers` Query Match করে এবং `sortUsers` Original Array Mutate না করে Copy Sort করে।

JAVASCRIPT

```javascript
// app.mjs — important fragment
async function loadUsers() {
  controller?.abort();
  controller = new AbortController();
  const currentRequest = ++requestNumber;
  reload.disabled = true;
  status.textContent = 'Loading users…';
  try {
    users = await fetchUsers({ signal: controller.signal });
    if (currentRequest === requestNumber) render();
  } catch (error) {
    if (currentRequest !== requestNumber || error.name === 'AbortError') return;
    users = [];
    list.replaceChildren();
    status.textContent = 'Could not load users. Check connection and retry.';
    console.error(error);
  } finally {
    if (currentRequest === requestNumber) reload.disabled = false;
  }
}
```

Security, Accessibility ও UX

- `textContent` দিয়ে User Data Render করা হয়েছে; Raw `innerHTML` ব্যবহার করা হয়নি।
- `role="status"` ও `aria-live="polite"` Loading/Result Announcements-এ সাহায্য করে।
- `AbortController` দিয়ে পুরোনো Request Cancel করা হয়; Request Counter Stale Result ঠেকায়।
- Network Error হলে Clear Message দেখায়; Reload Button আবার চেষ্টা করতে দেয়।
- Browser Fetch ও JSONPlaceholder API-এর জন্য Network Access দরকার; Offline হলে Failure Message স্বাভাবিক।

১০.১২ Common Mistakes ও Debugging

ভুল	কারণ	সমাধান
`await` ছাড়া Promise Log	Result এখনো Pending	`await` বা `.then()`
HTTP 404-কে `catch` ধরবে ভাবা	`fetch` সাধারণত HTTP Error-এ Reject করে না	`response.ok` Check
`forEach(async ...)` Await করা	`forEach` Promise Aggregate করে না	`for...of` বা `Promise.all`
Loading State না দেখানো	User মনে করে App আটকে গেছে	Status Text/Spinner
পুরোনো Response নতুন Result Overwrite	Request Race	Abort + Request-ID
Unbounded Concurrent Requests	API/Browser Overload	Limit Concurrency
`innerHTML`-এ API Data	XSS Risk	`textContent`/DOM Nodes
Timer Delay-কে Exact Timing ধরা	Event Loop Scheduling	Delay-কে Minimum Treat

১০.১৩ Exercises

1. `setTimeout(...,0)` কেন Synchronous Statement-এর পরে চলে—ব্যাখ্যা করো।
2. একটি Callback-based `readLater(value, callback)` Function লেখো।
3. `Promise.resolve(5)`-কে Chain করে 20 Output করো।
4. `async/await` দিয়ে দুটি Sequential Promise Resolve করো।
5. Independent দুটি Request Concurrent করার Code লেখো।
6. `Promise.allSettled()` দিয়ে Partial Failure Handle করো।
7. `fetch()`-এর HTTP 404 Error কীভাবে Handle করবে?
8. `AbortController` দিয়ে User Search-এর পুরোনো Request Cancel করো।
9. `filterUsers()`-এ Search Case-Insensitive করো।
10. Project-এ Empty Result-এর জন্য আলাদা Message যোগ করো।

১০.১৪ Exercise Solutions

১. Timer Callback একটি পরবর্তী Task হিসেবে Queue-তে যায়; বর্তমান Call Stack শেষ হওয়ার আগে চলতে পারে না।

২.

```javascript
function readLater(value, callback) {
  setTimeout(() => callback(null, value), 100);
}
readLater('done', (error, value) => console.log(error ?? value));
```

৩.

```javascript
Promise.resolve(5).then(n => n * 4).then(console.log); // 20
```

৪.

```javascript
async function sequential() {
  const a = await Promise.resolve(3);
  const b = await Promise.resolve(4);
  return a + b;
}
sequential().then(console.log); // 7
```

৫.

```javascript
const [a, b] = await Promise.all([fetch('/api/a'), fetch('/api/b')]);
```

৬.

```javascript
const results = await Promise.allSettled([Promise.resolve(1), Promise.reject('offline')]);
for (const result of results) console.log(result.status, result.status === 'fulfilled' ?
result.value : result.reason);
```

৭.

```javascript
const response = await fetch('/missing');
if (!response.ok) throw new Error(`HTTP ${response.status}`);
```

৮. নতুন Request শুরু করার আগে `controller?.abort()` এবং নতুন `AbortController` তৈরি করো; `fetch(url, { signal: controller.signal })` ব্যবহার করো। Project-এর `app.mjs`-এ পূর্ণ Example রয়েছে।

৯.

```javascript
const match = user.name.toLocaleLowerCase().includes(query.toLocaleLowerCase());
```

১০. `render()`-এ `filtered.length === 0` হলে `status.textContent = 'No matching users found'` দেখাও; List Empty রাখো।

১০.১৫ Knowledge Check — ১০টি MCQ

Q1. `async` Function কী Return করে? (A) Number (B) Promise (C) Timer (D) Event — **উত্তর: B**। Return Value Promise দিয়ে Wrap হয়।

Q2. `fetch()` HTTP 404 পেলে সাধারণত কী হয়? (A) Reject (B) Fulfill with Response (C) Crash (D) Retry — **উত্তর: B**। `response.ok` Check করতে হয়।

Q3. Promise Reaction সাধারণত কোথায় Schedule হয়? (A) Call Stack immediately (B) Microtask Queue (C) CSS Queue (D) Storage — **উত্তর: B**। Current Stack শেষ হওয়ার পরে Microtask চলে।

Q4. `Promise.all()`-এ একটি Promise Reject হলে কী হয়? (A) সব Cancel (B) Aggregate Reject (C) সব Ignore (D) Browser Close — **উত্তর: B**। অন্য Operations স্বয়ংক্রিয় Cancel হয় না।

Q5. `await` কী Pause করে? (A) Entire Browser (B) Current Async Function Continuation (C) CSS (D) Server — **উত্তর: B**। Thread পুরোপুরি Block হয় না।

Q6. Abort Signal কোন API-তে দেওয়া যায়? (A) `fetch` (B) `JSON.stringify` (C) `Math.random` (D) `parseInt` — **উত্তর: A**। Request Cancellation-এ সাহায্য করে।

Q7. সব Promise Settled হওয়ার Result পেতে কোনটি? (A) `allSettled` (B) `race` (C) `any` (D) `resolve` — **উত্তর: A**। Success ও Failure দুটোই Report করে।

Q8. API Data DOM-এ Text হিসেবে নিরাপদে দেখাতে কোনটি? (A) `innerHTML` (B) `eval` (C) `textContent` (D) `document.write` — **উত্তর: C**। Text HTML হিসেবে Parse হয় না।

Q9. Independent Requests দ্রুত শুরু করতে কোনটি? (A) Sequential `await` (B) `Promise.all` (C) `while(true)` (D) `alert` — **উত্তর: B**। Concurrent Operations-এর জন্য।

Q10. Stale Response Problem-এর সমাধান কী? (A) Ignore errors (B) Abort + Request ID (C) More timers (D) `var` — **উত্তর: B**। পুরোনো Request নতুন UI Overwrite করতে পারে না।

১০.১৬ Chapter Summary ও Next Steps

এই অধ্যায়ে Async Execution-এর Mental Model, Promise Chaining, `async/await`, Microtask Scheduling, HTTP Response Handling, Cancellation, Concurrency এবং User-Friendly Async UI তৈরি করা শিখেছ। Project Source ও Unit Tests আলাদা Files-এ রয়েছে। **পরবর্তী অধ্যায়: Error Handling, Debugging এবং Defensive Programming।**

Further Reading

- MDN — JavaScript Guide: Using Promises: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Using_promises
- MDN — `async function`: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function
- MDN — Fetch API: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- MDN — AbortController: https://developer.mozilla.org/en-US/docs/Web/API/AbortController
- MDN — Event Loop: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Execution_model